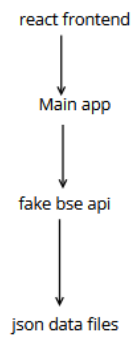
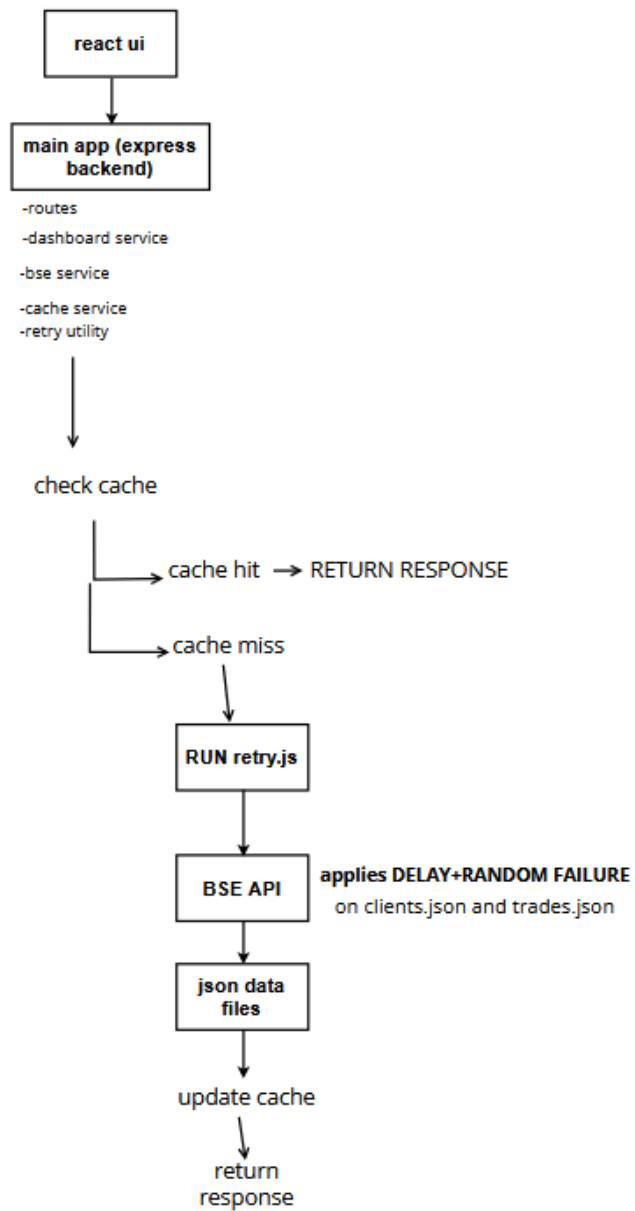


1. MAIN ARCHITECTURE:



2. REQUEST FLOW



3. BSE API ARCHITECTURE



4. ARCHITECTURE REASONING:

- **Frontend (React + Vite)** provides the user interface and communicates with the backend using REST APIs.
- **Main Application (Express.js)** handles all business logic, data aggregation, and API orchestration.
- **Service Layer** (bseService, dashboardService, cacheService) separates business logic from route handlers, improving maintainability.
- **Retry Utility** retries failed requests to the Fake BSE API, improving resilience against transient failures.
- **Cache Layer** stores aggregated dashboard data in dashboard.json to reduce repeated API calls and improve response times.
- **Fake BSE API** simulates an external third-party service by serving data from JSON files instead of a database.
- **Delay Middleware** introduces configurable latency and random failures to emulate real-world API behavior and validate retry handling.

- **JSON Data Files** (employees.json, clients.json, mappings.json, trades.json) act as the application's data source for the simulated BSE service.
- **Modular Structure** (routes, services, middleware, utilities) keeps responsibilities separated, making the project easier to maintain and extend.

5. Running at 100× Data Volume

To support 100× more clients and trades, the application can be scaled by

- replacing the current data storage with a production database such as PostgreSQL,
- using Redis for shared caching,
- and moving BSE data synchronization to background workers.
- Large datasets should be served using pagination and indexed queries to improve performance.
- real-time technologies such as WebSockets or Server-Sent Events (SSE) can replace polling for more efficient updates.

These changes would allow the application to maintain fast response times and reliable data synchronization even with significantly larger data volumes.